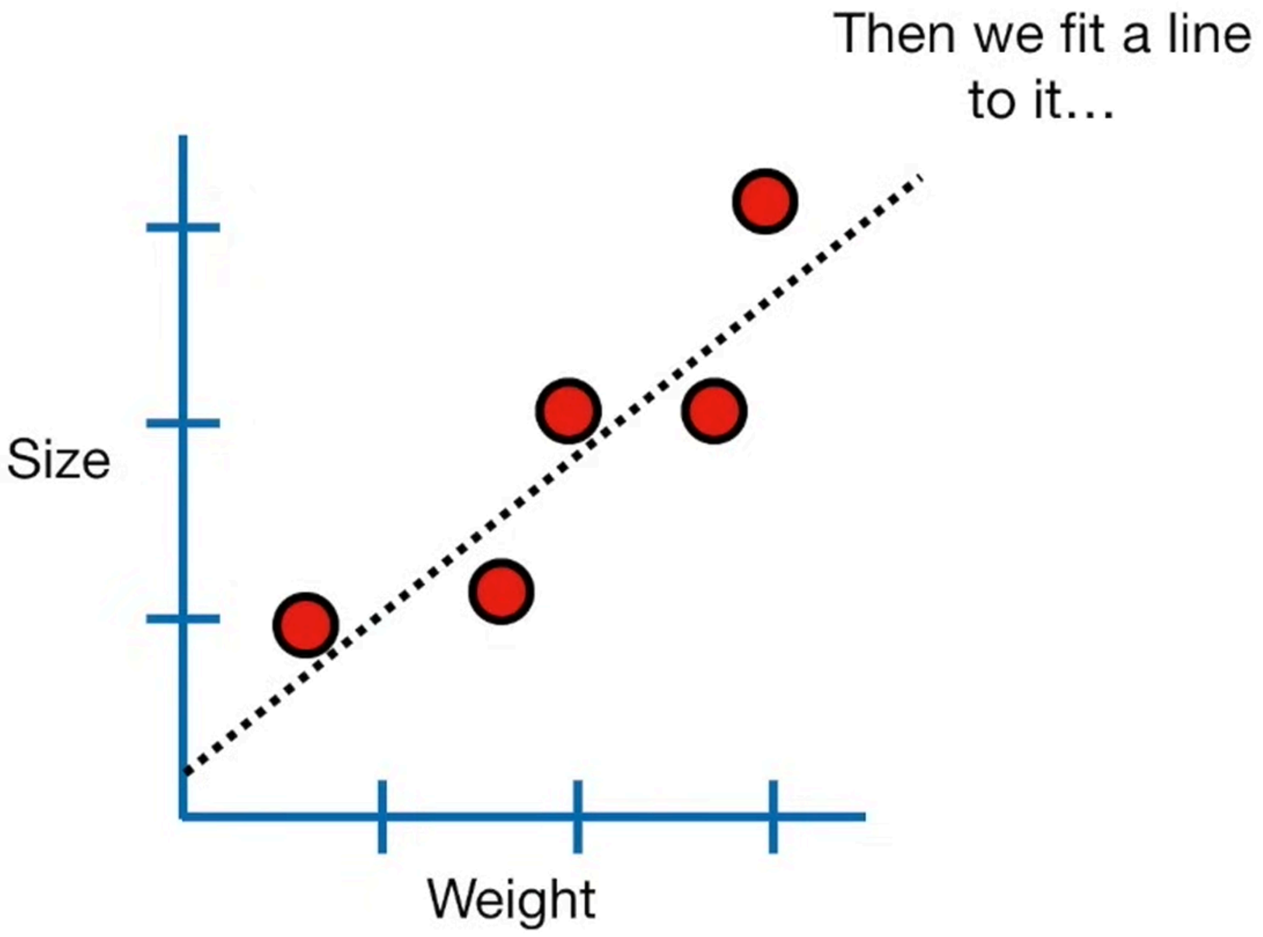
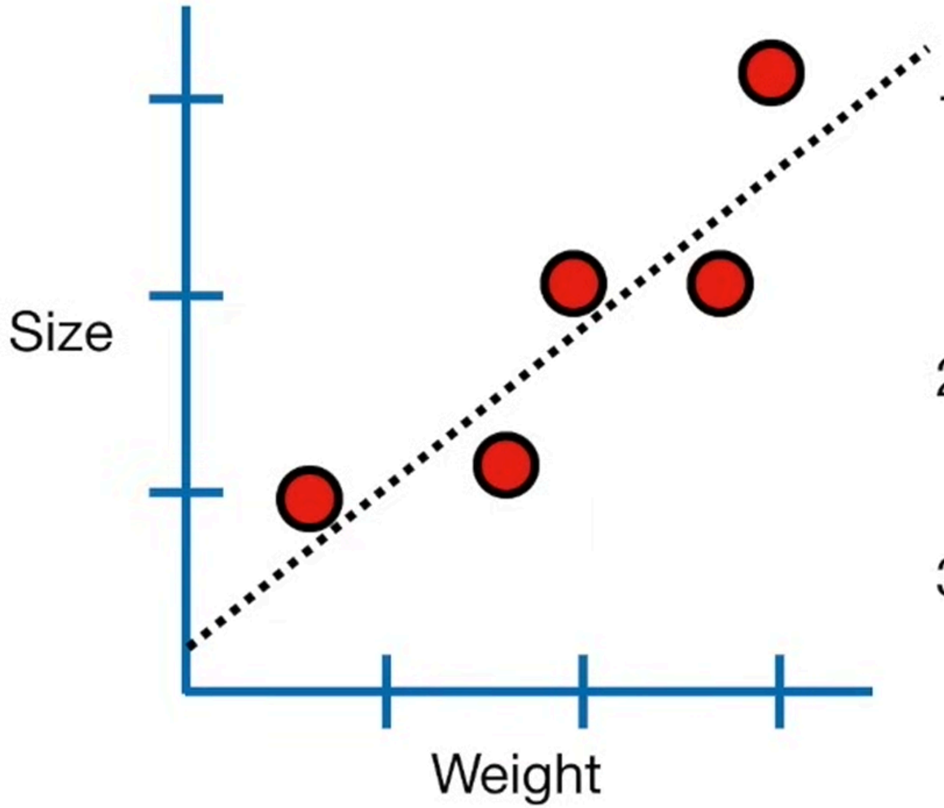
Before we dive into logistic regression, let's take a step back and review linear regression.



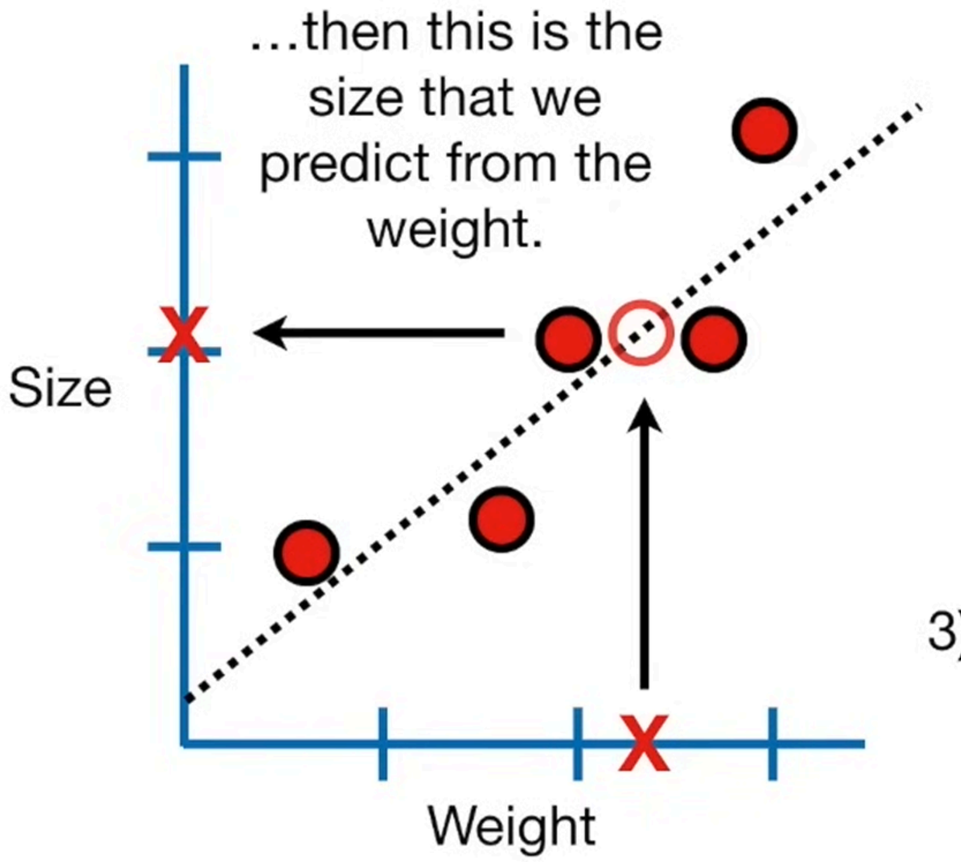


…and with that line, we could do a lot of things:

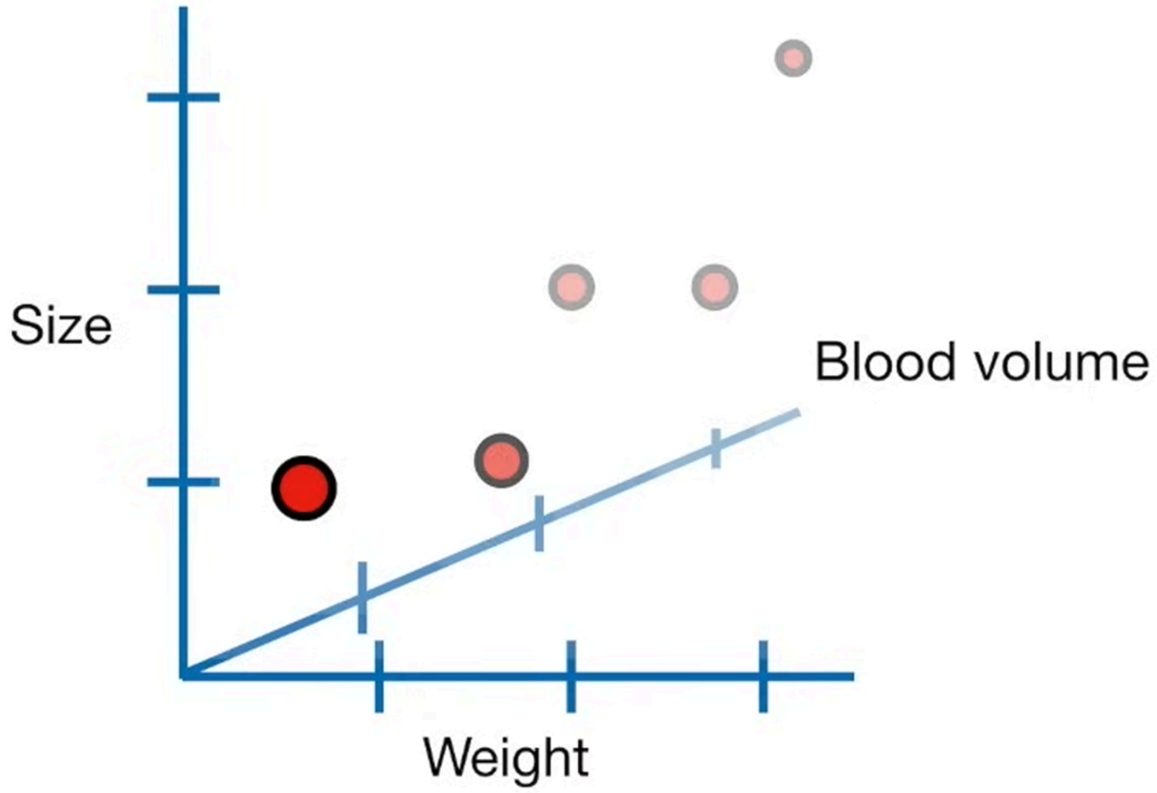
1) Calculate R^2 and determine if ***weight*** and ***size*** are correlated. Large values imply a large effect.

2) Calculate a p-value to determine if the R^2 value is statistically significant.

3) Use the line to predict ***size*** given ***weight***.

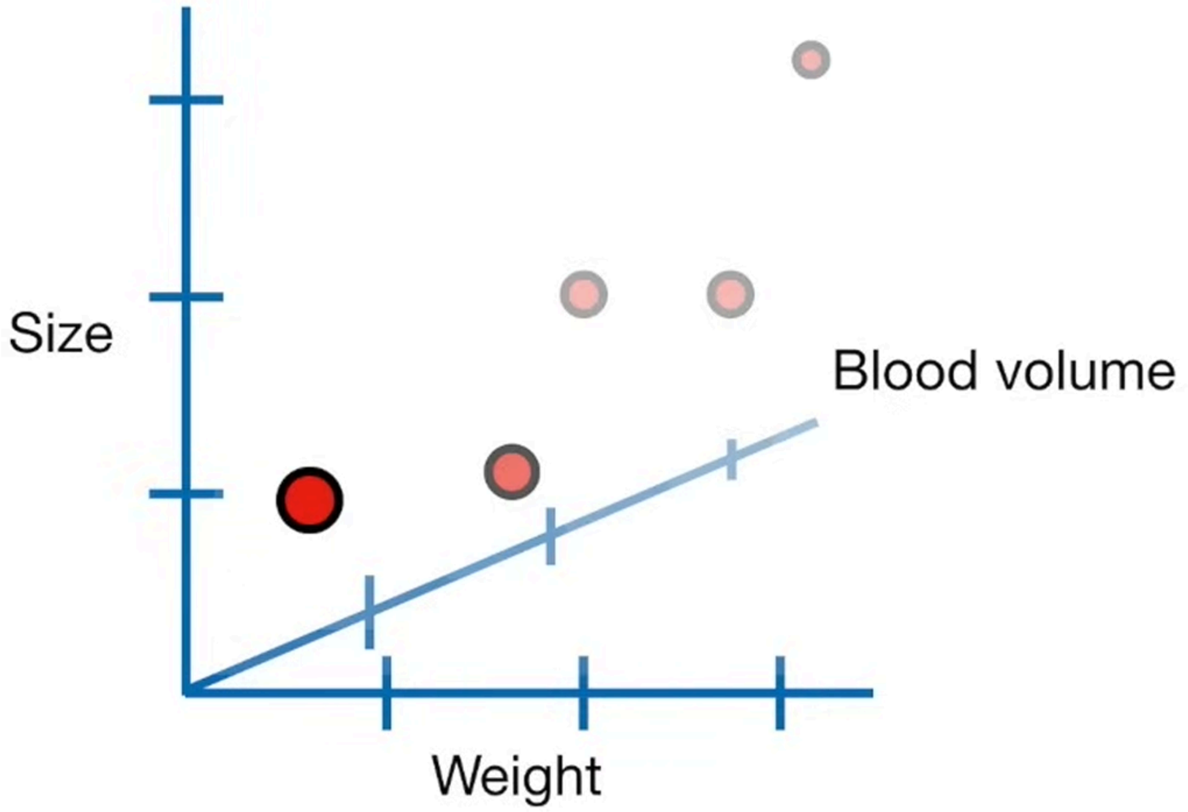


3) Use the line to predict ***size*** given ***weight***.



Now we are trying to predict ***size*** using ***weight*** and ***blood volume***.

Alternatively, we could say we are trying to model ***size*** using ***weight*** and ***blood volume***.



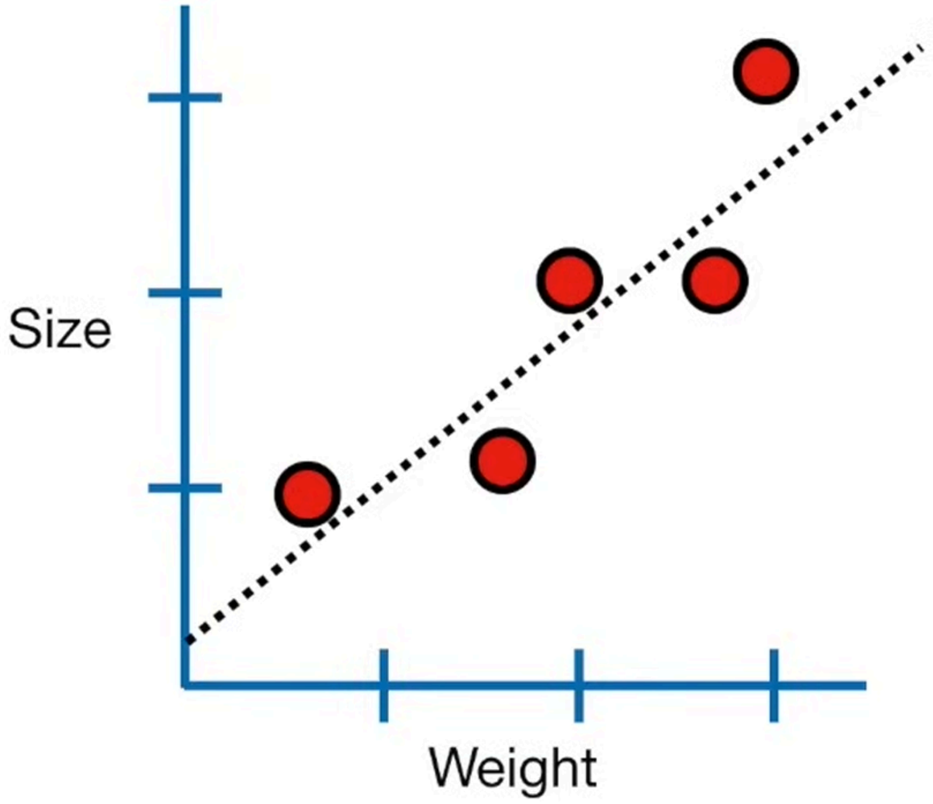
Multiple regression did the same things that normal regression did…

1) Calculate R^2.

2) Calculate the p-value.

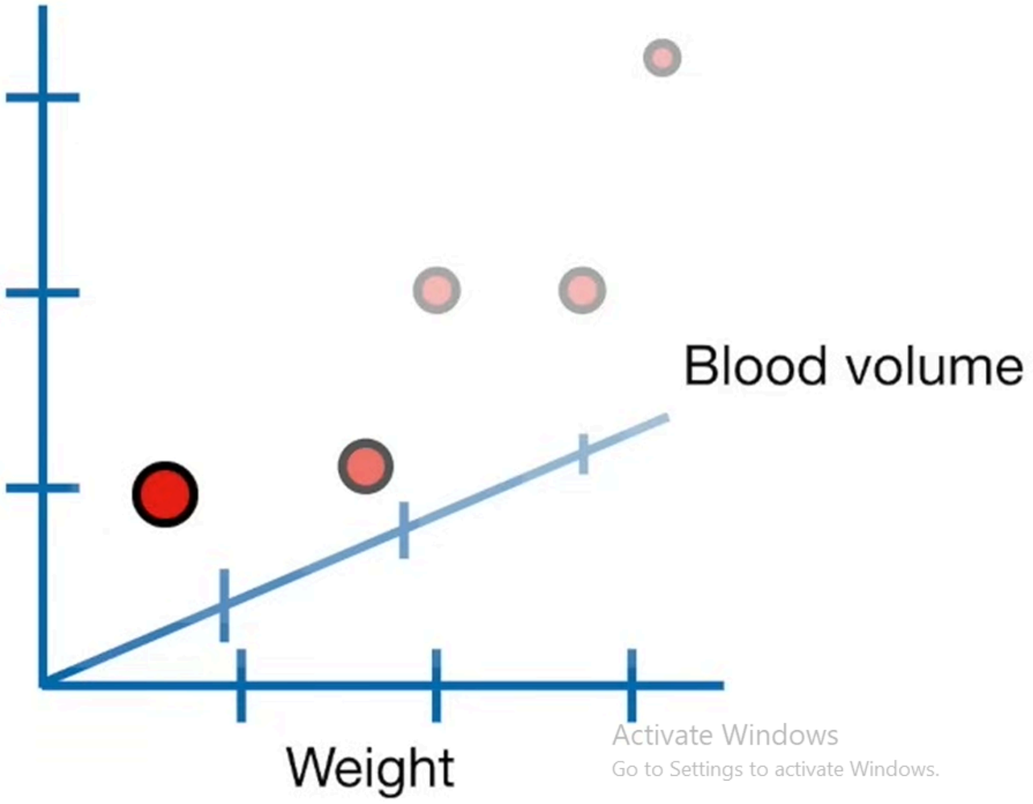
3) Predict ***size*** given ***weight*** and ***blood volume***.

Normal regression, using ***weight*** to predict **size**.

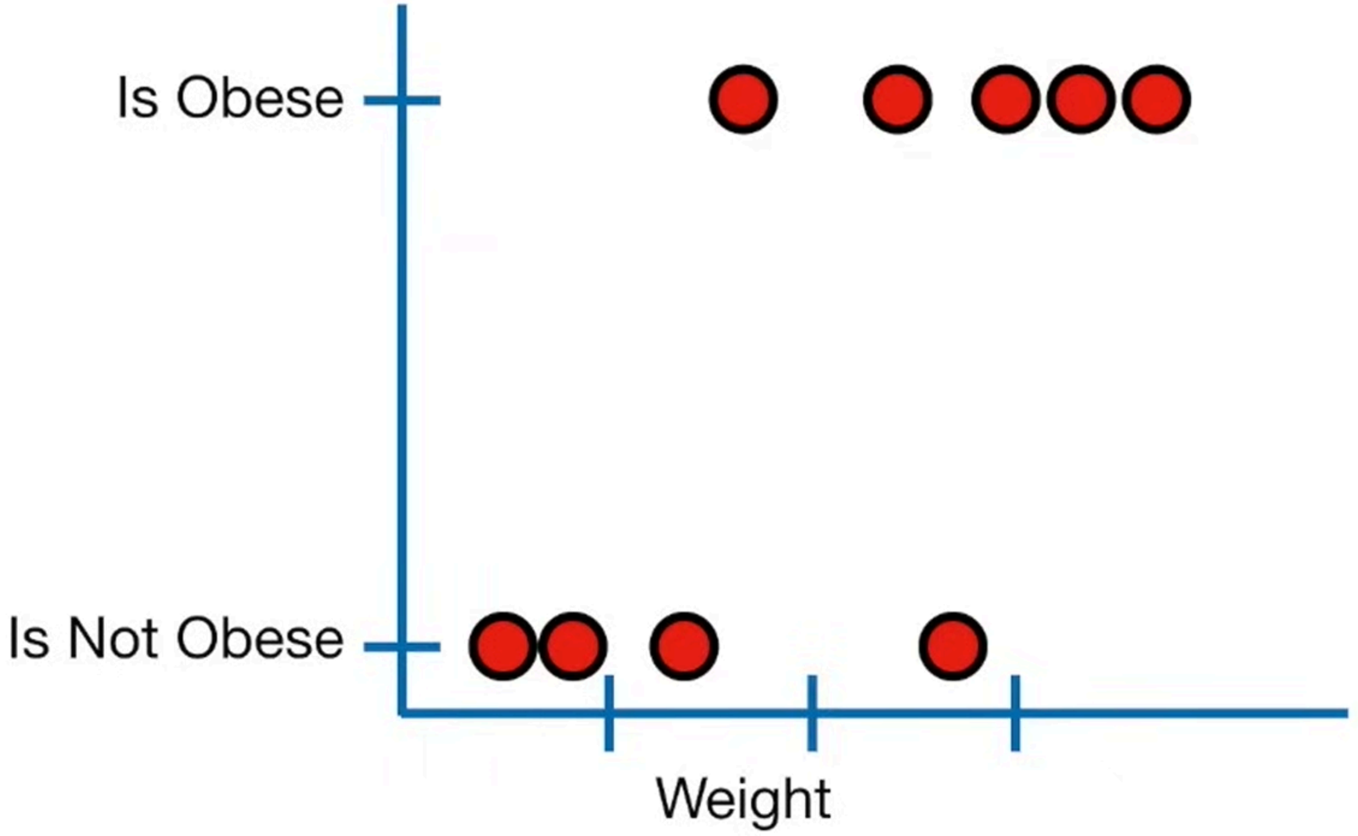


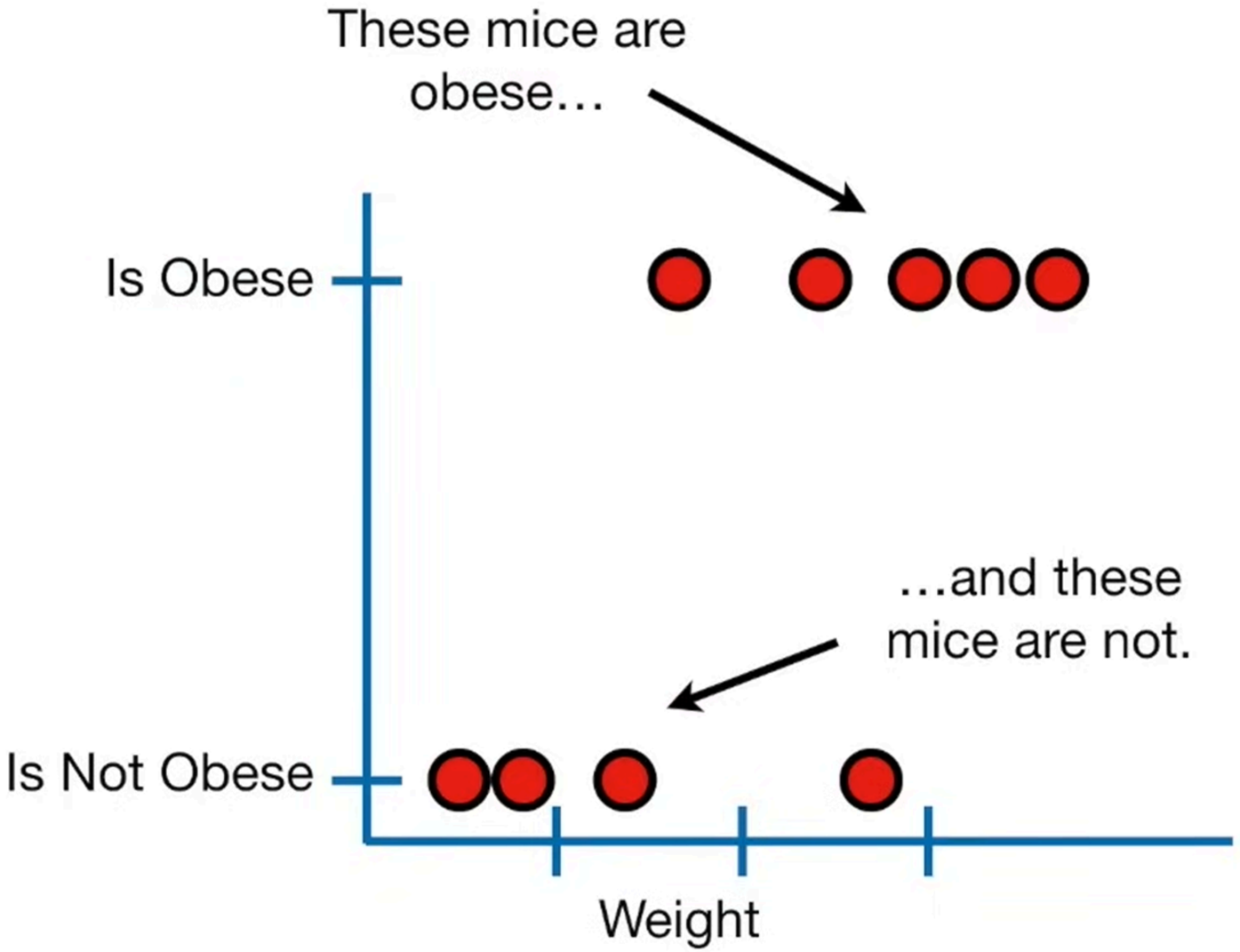
VS

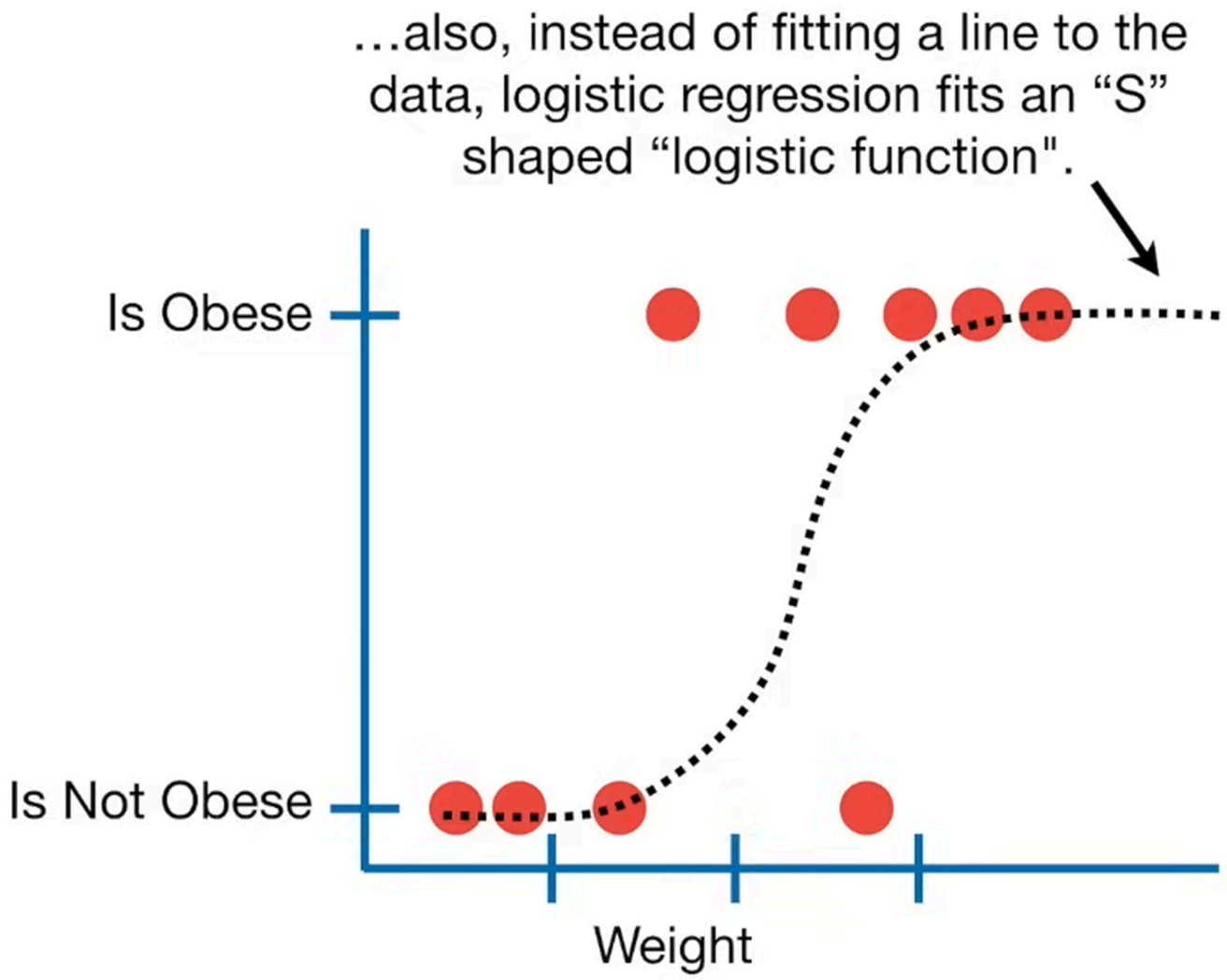
Multiple regression, using ***weight*** and ***blood volume*** to predict ***size***.



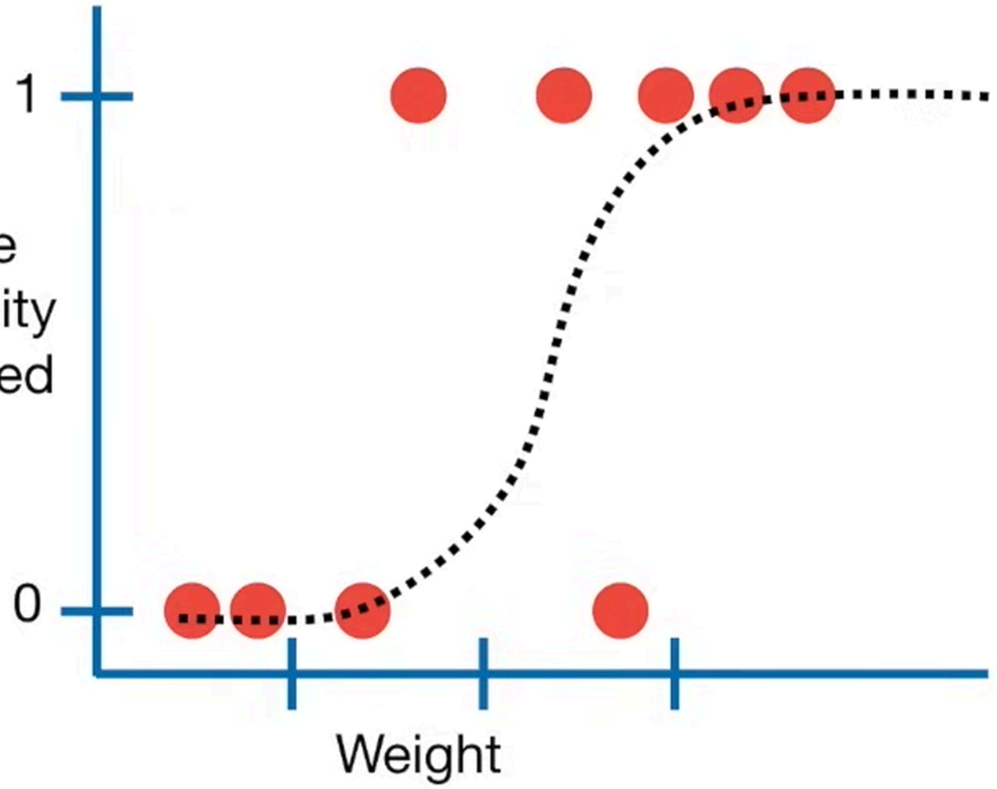
Logistic regression is similar to linear regression, except…

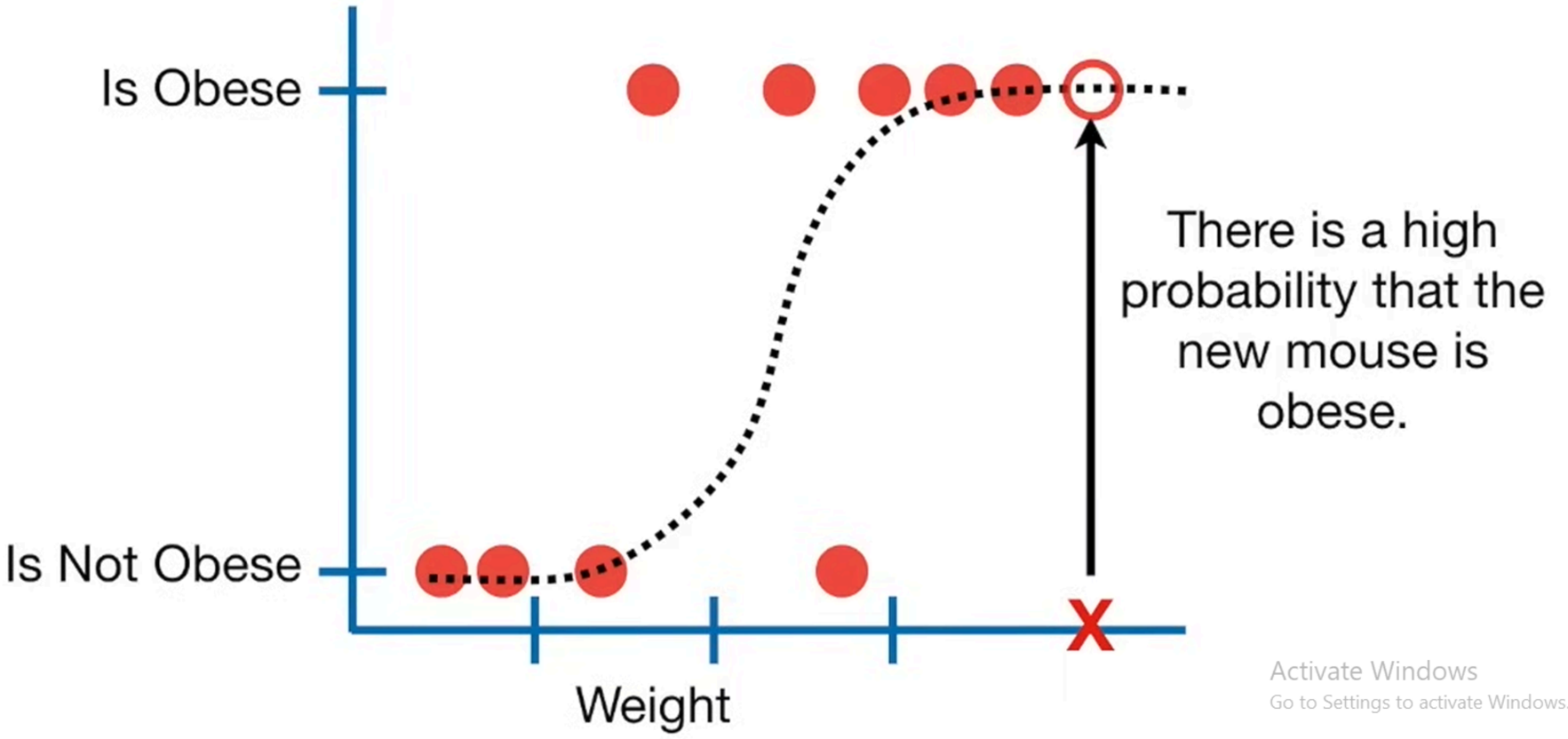


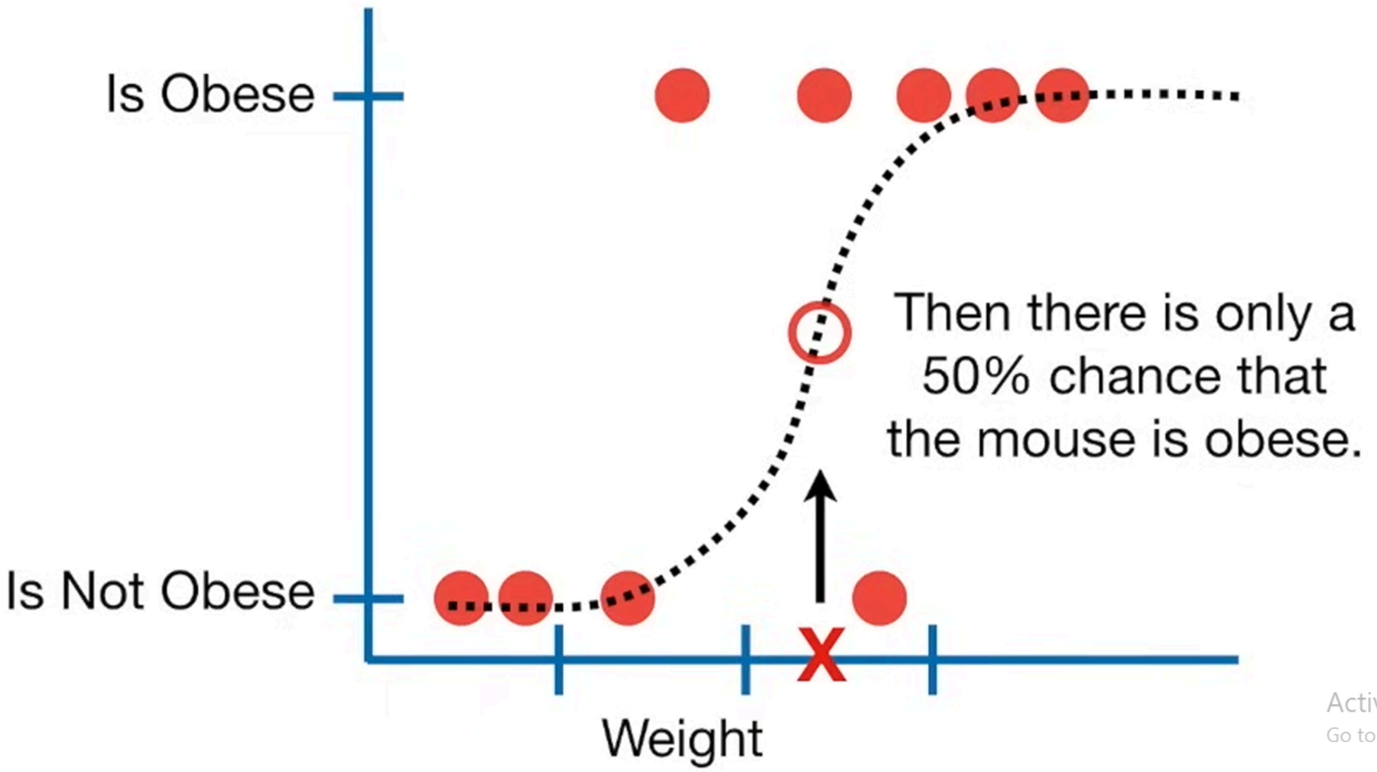


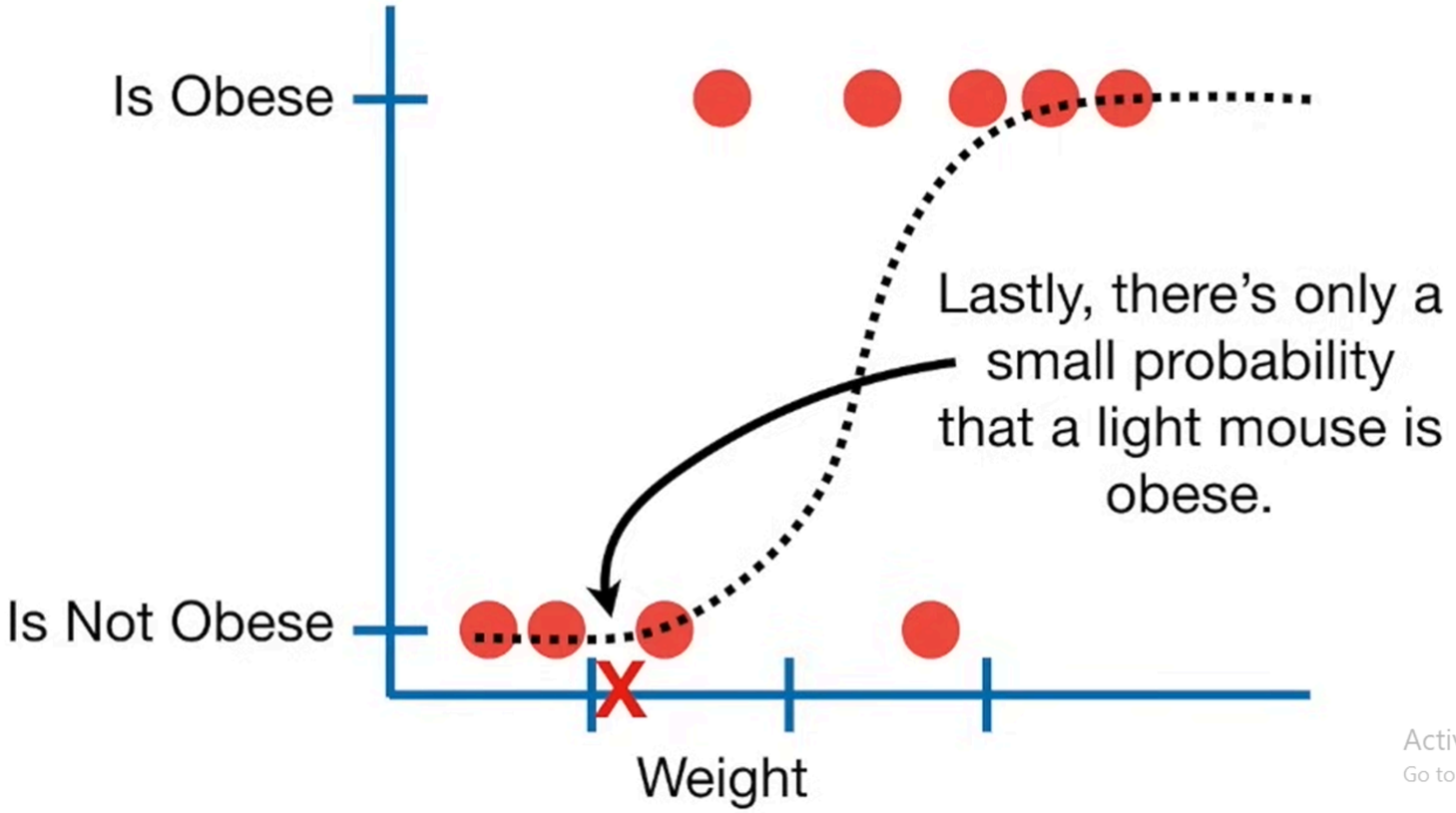


…and that means that the curve tells you the probability that a mouse is ***obese*** based on its ***weight***.

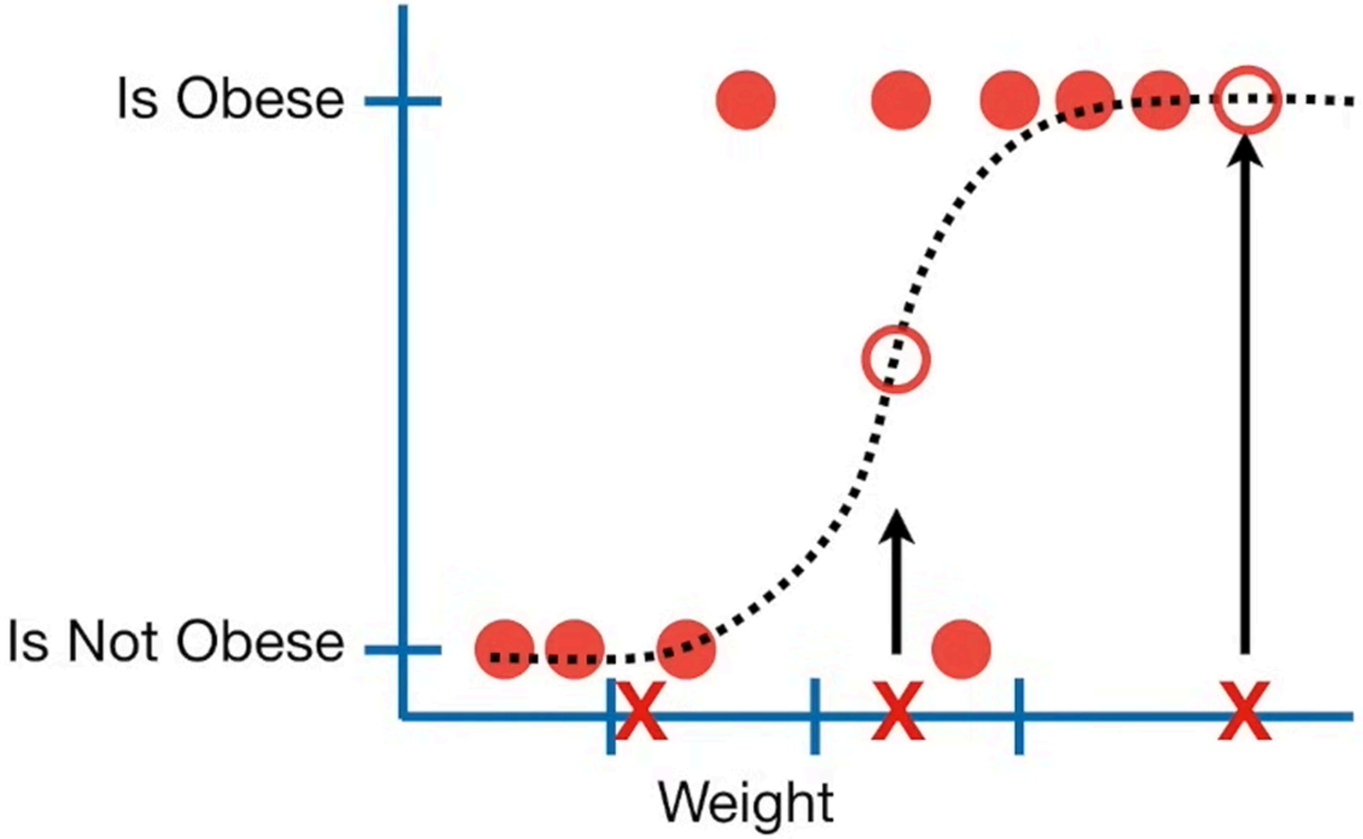




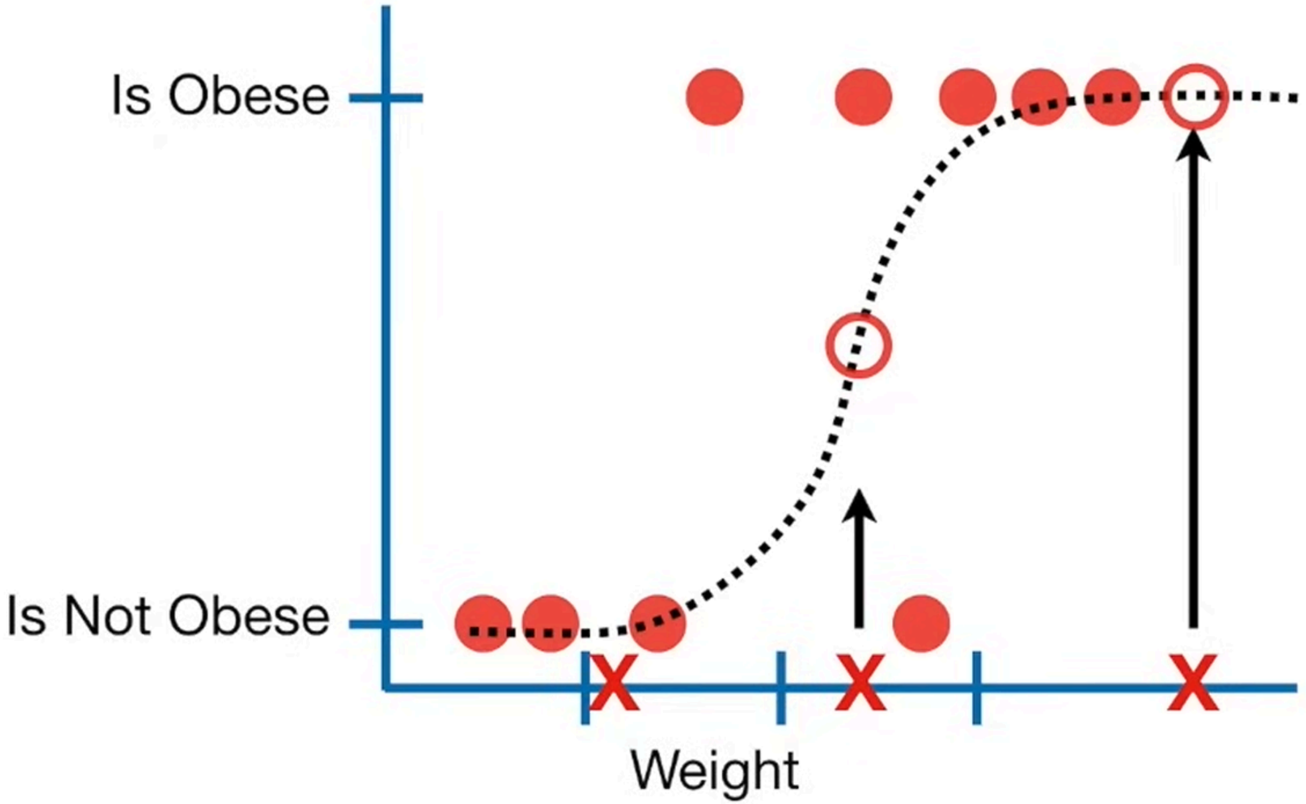




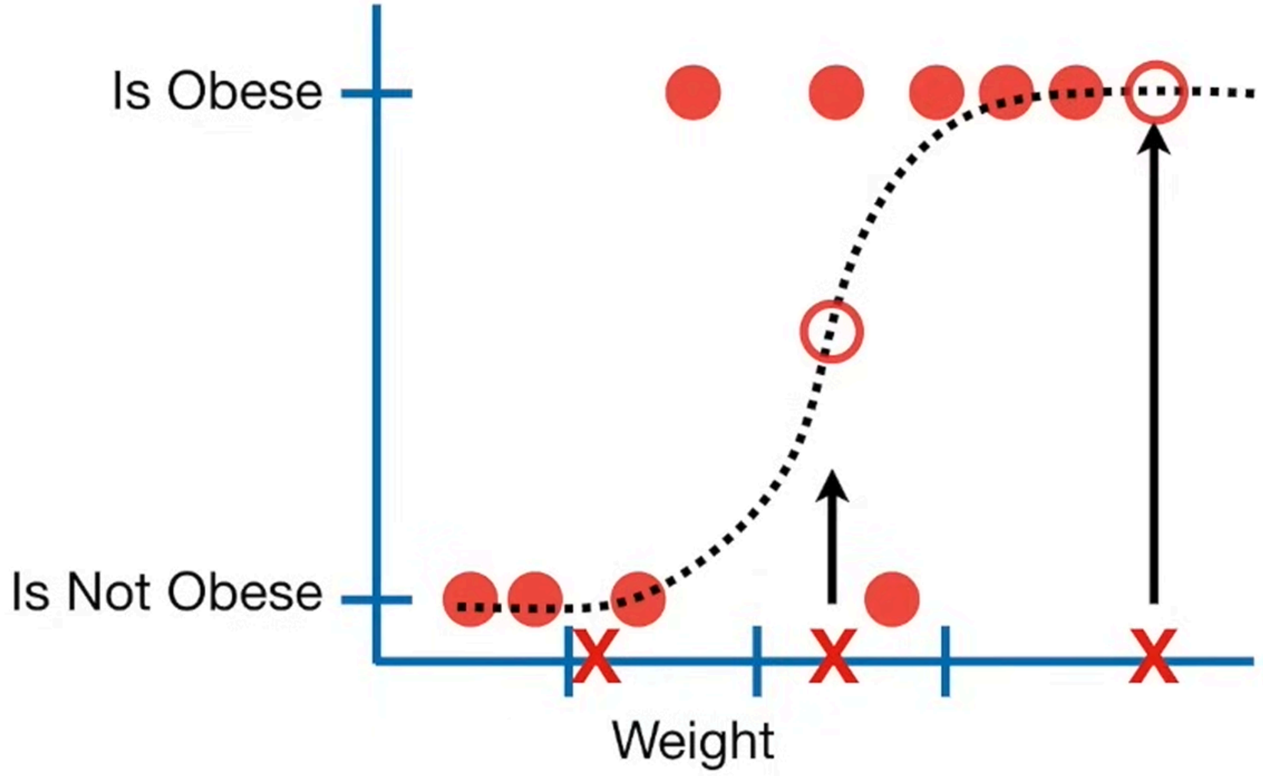
Although logistic regression tells the probability that a mouse is obese or not, it's usually used for classification.



For example, if the probability a mouse is obese is > 50%, then we'll classify it as obese, otherwise we'll classify it as "not obese".



Logistic regression's ability to provide probabilities and classify new samples using continuous and discrete measurements makes it a popular machine learning method.



Linear Regression

1. Home prices
2. Weather
3. Stock price

Predicted value is continuous

1. Email is spam or not
2. Will customer buy life insurance?
3. Which party a person is going to vote for?
    1. Democratic
    2. Republican
    3. Independent

Classification Types

Will customer buy life insurance?

1. Yes
2. No

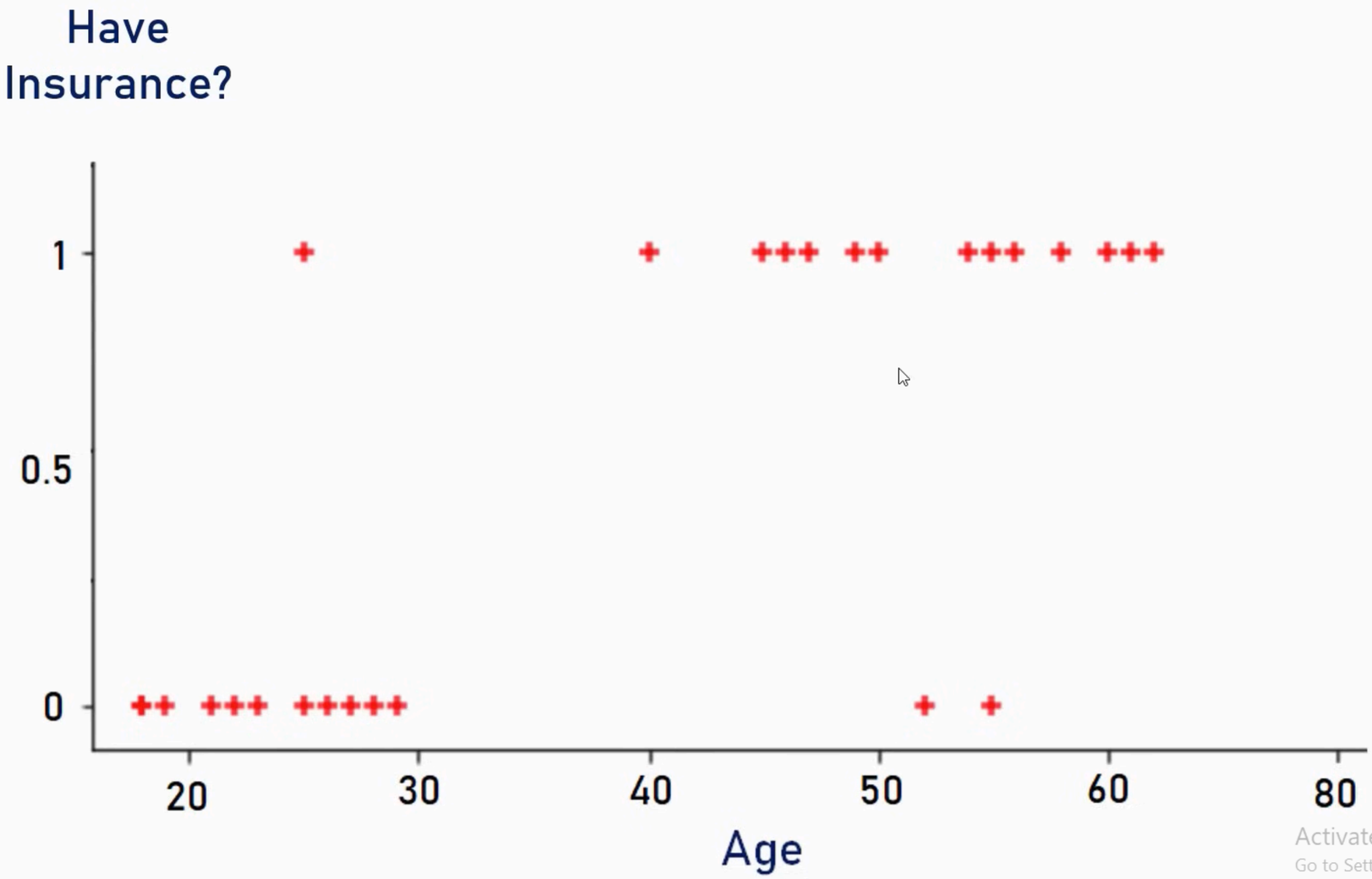
Binary Classification

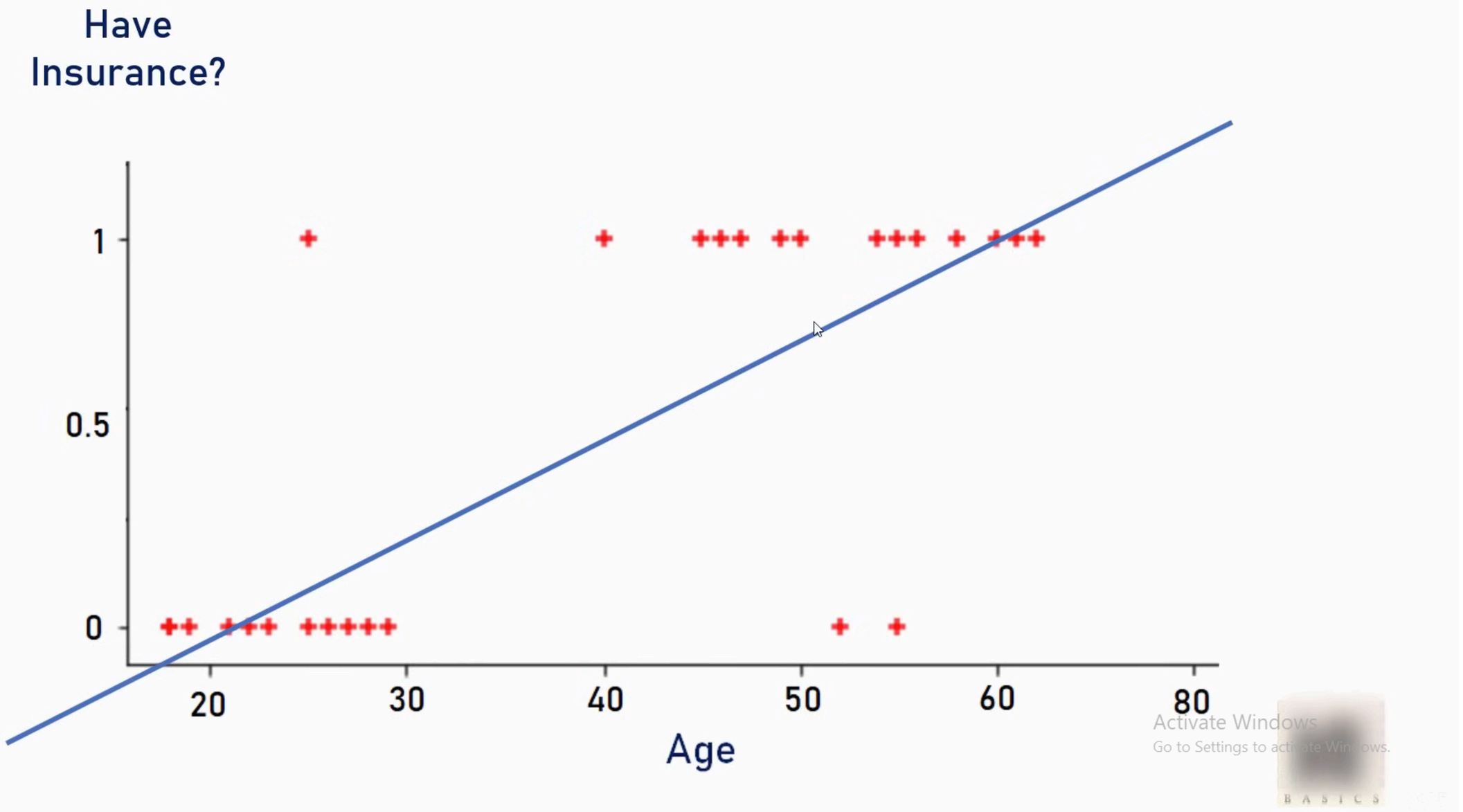
Which party a person is going to vote for?

1. Democratic
2. Republican
3. Independent

Multiclass Classification

age	have_insurance
22	0
25	0
47	1
52	0
46	1
56	1
55	0
60	1
62	1
61	1
18	0
28	0
27	0
29	0
49	1





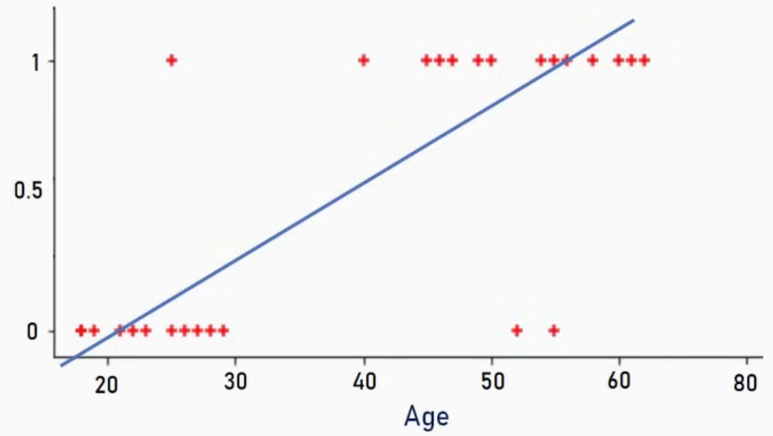
$$sigmoid(z) = \frac{1}{1 + e^{-z}}$$

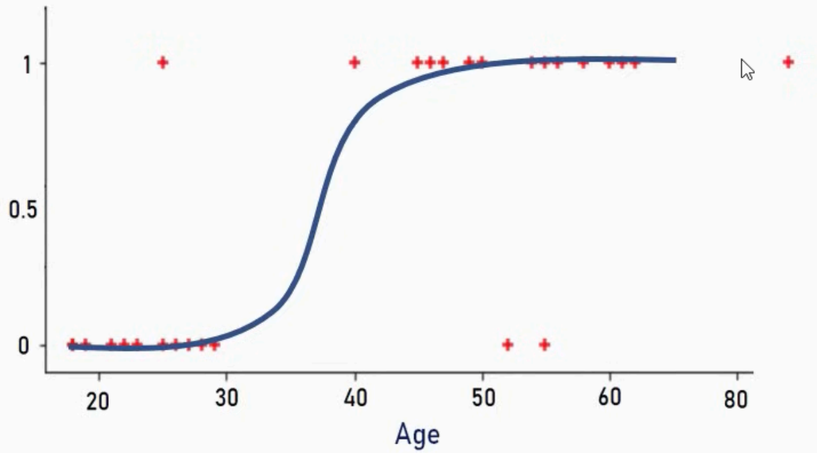
e = Euler's number ~ 2.71828

Sigmoid function converts input into range 0 to 1

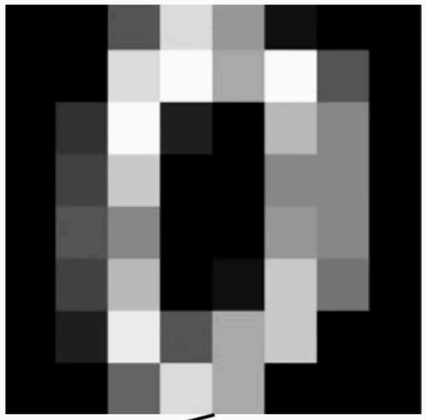
$$y = m * x + b$$

$$y = \frac{1}{1 + e^{-(m*x+b)}}$$





Identify hand written digits recognition



0 1 2 3 4 5 6 7 8

Logistic Regression Algorithm (Step-by-Step)

Step 1: Initialize Parameters

- Initialize weights w1,w2,wn
- Initialize bias b
- Usually set to 0 or small random values.

Step 2: Compute Linear Combination

For each training example:

$z=wX+b$

Step 3: Apply Sigmoid Function

Convert z into probability:

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

Now:

- If $\hat{y}$ close to 1 → Class 1
- If $\hat{y}$ close to 0 → Class 0

Step 4: Compute Loss (Cost Function)

Logistic Regression uses **Binary Cross-Entropy Loss**:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

For entire dataset:

$$J = \frac{1}{m} \sum L$$

$$\frac{\partial J}{\partial w} = \frac{dJ}{dz} \cdot \frac{\partial z}{\partial w} = (\hat{y} - y)x$$

$$\frac{\partial J}{\partial b} = \frac{dJ}{dz} \cdot \frac{\partial z}{\partial b} = (\hat{y} - y)$$

Step 5: Compute Gradients

Calculate derivatives of loss with respect to:

- Weights w
- Bias b

$$\frac{\partial J}{\partial w}$$

$$\frac{\partial J}{\partial b}$$

Step 6: Update Parameters (Gradient Descent)

$$w = w - \alpha \frac{\partial J}{\partial w}$$

$$b = b - \alpha \frac{\partial J}{\partial b}$$

Step 7: Repeat Until Convergence

Repeat Steps 2–6 until:

- Loss becomes very small
- Or maximum iterations reached

Step 8: Make Final Prediction

After training:

$$\hat{y} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{if } \hat{y} < 0.5 \end{cases}$$

full example live update

✅ Rule We Will Use

1. Compute

$$z = wx + b$$

2. Compute sigmoid

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

3. Prediction rule:

   If $\hat{y} \geq 0.5$ → predict 1

   If $\hat{y} < 0.5$ → predict 0

4. If prediction == actual

   → Loss = 0

   → No update

5. If prediction ≠ actual

   → Compute gradient

   → Update

📊 Dataset (10 points)

x	y
18	0
22	0
25	0
28	0
30	0
35	1
38	1
42	1
45	1
50	1

🔹 Initialize

$$w = 0$$

$$b = 0$$

$$\alpha = 0.01$$

$$z = wx + b$$

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

🔴 Point 1: (18,0)

Step 1: Linear

$$z = 0$$

Step 2: Sigmoid

$$\hat{y} = 0.5$$

Step 3: Prediction

0.5 ≥ 0.5 → predict = 1

Actual = 0

❌ Wrong

Update

$$\frac{\partial J}{\partial w} = (0.5 - 0)(18) = 9$$

$$\frac{\partial J}{\partial b} = 0.5$$

$$w = 0 - 0.01(9)$$

$$w = -0.09$$

$$b = 0 - 0.01(0.5)$$

$$b = -0.005$$

🔴 Point 2: (22,0)

Current:

$$w = -0.09$$

$$b = -0.005$$

Forward

$$z = (-0.09)(22) - 0.005 = -1.985$$

$$\hat{y} \approx 0.121$$

Prediction

0.121 < 0.5 → predict = 0

Actual = 0

✅ Correct

According to your rule:

Loss = 0

No update

So:

$$w = -0.09$$

$$b = -0.005$$

🔴 Point 3: (25,0)

Forward

$$z = (-0.09)(25) - 0.005$$

$$z = -2.255$$

$$\hat{y} \approx 0.095$$

Prediction = 0

Actual = 0

✅ Correct

No update.

🔴 Point 4: (28,0)

$$z = (-0.09)(28) - 0.005$$

$$z = -2.525$$

$$\hat{y} \approx 0.074$$

Prediction = 0

Actual = 0

✅ No update.

🔵 Point 6: (35,1)

Forward

$$z = (-0.09)(35) - 0.005$$

$$z = -3.155$$

$$\hat{y} \approx 0.041$$

Prediction = 0

Actual = 1

❌ Wrong

Update

$$\frac{\partial J}{\partial w} = (0.041 - 1)(35)$$

$$= (-0.959)(35)$$

$$= -33.565$$

$$\frac{\partial J}{\partial b} = -0.959$$

$$w = -0.09 - 0.01(-33.565)$$

$$w = -0.09 + 0.33565$$

$$w = 0.24565$$

$$b = -0.005 - 0.01(-0.959)$$

$$b = 0.00459$$

🎯 Final Parameters After One Pass

$$w \approx 0.24565$$

$$b \approx 0.00459$$

We will use the **final parameters** from the last step:

$$w = 0.24565, \quad b = 0.00459$$

Decision rule:

$$\hat{y} \geq 0.5 \Rightarrow \text{predict } 1$$

$$\hat{y} < 0.5 \Rightarrow \text{predict } 0$$

Check overall loss and then loss is minimum not need to repeat the process
if not we have to repeat the process multiple time

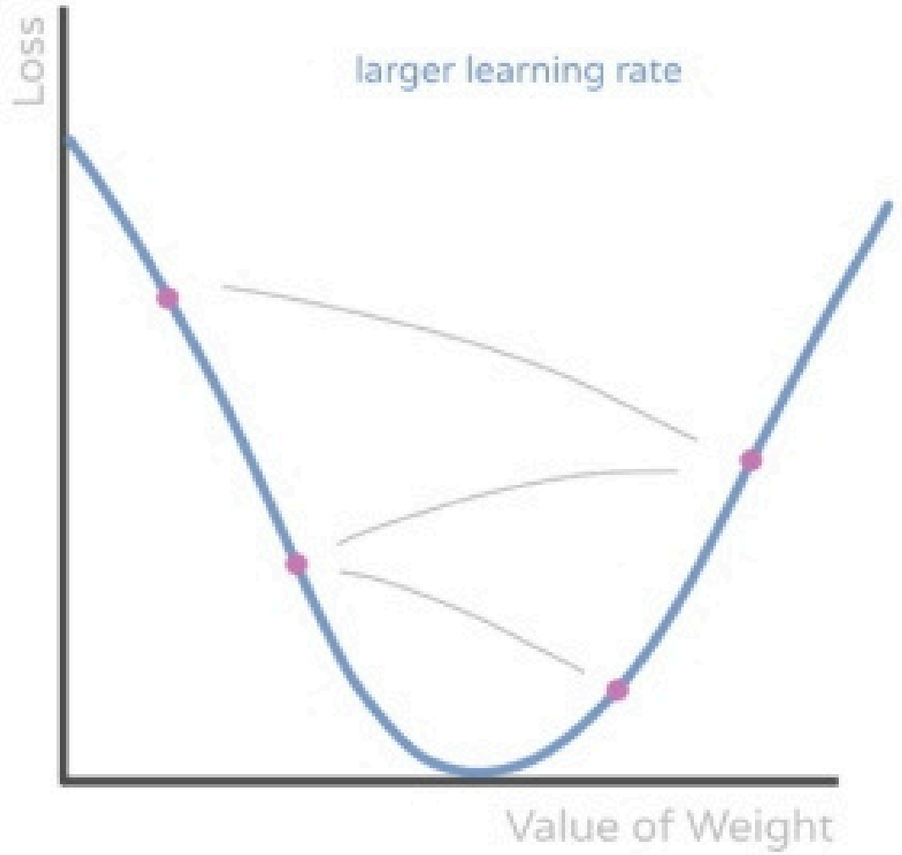
Epoch =1

Each data shown exactly one time at traning

so for minimizing the loss we do a several epoch and repeat entire process

Gradient Descent

aicorr.com



🔹 Point 1: x = 20

$$z = 0.24565 * 20 + 0.00459$$

$$z = 4.913 + 0.00459 = 4.9176$$

$$\hat{y} = \frac{1}{1 + e^{-4.9176}}$$

$$e^{-4.9176} \approx 0.0073$$

$$\hat{y} \approx \frac{1}{1 + 0.0073} \approx 0.993$$

Prediction = 1

✅ Even at age 20, probability is already very high due to strong weight from positive point (35) update.

1 The Three Flavors of Gradient Descent

Type	When we update	Loss used	Pros / Cons
SGD (Stochastic Gradient Descent)	After **each data point**	Loss of that **single point only**	Very fast, but noisy. Model can jump around a lot.
Batch Gradient Descent	After **entire dataset**	**Overall / average loss**	Very stable, but can be slow for large datasets.
Mini-Batch Gradient Descent	After **small batches** (e.g., 32 or 64 points)	Average loss for the batch	Best of both worlds: faster than batch more stable than SGD.